

Relatório de Análise de Segurança

API Backend — Sistema de Gestão de Academias
Flask + Supabase (PostgreSQL) · React/Next.js

Escopo	Endpoints da API + arquitetura de autenticação/autorização
Tipo	Análise estática (revisão de código)
Revisão	2 — pós-correção do achado A-1
Data	28/06/2026
Endpoints analisados	51 rotas em 10 blueprints
Base	backend/app/ (auth, alunos, instrutores, planos, mensalidades, avaliacoes, configuracoes, dashboard, relatorios, portal)

Documento técnico confidencial — uso interno do projeto.

1. Sumário Executivo

A aplicação apresenta um **nível de maturidade de segurança acima da média** para o estágio do projeto. A arquitetura de autenticação/autorização é sólida e foi claramente pensada: todas as 51 rotas (exceto o login público) exigem autenticação, a autorização por papel é aplicada via decorators, e os pontos clássicos de vazamento horizontal (BOLA/IDOR) nas leituras sensíveis por ID foram fechados com RLS por identidade do usuário.

A análise identificou **1 falha concreta e explorável** (IDOR na exportação de PDF de avaliações físicas) — **já corrigida e coberta por testes de regressão** nesta revisão. Permanecem itens de hardening relevantes para o deploy em produção que se aproxima (rate limit não compartilhado entre workers e token em localStorage com CSP ainda em modo de observação). Não foram encontradas falhas críticas de injeção, exposição de segredos ou bypass de autenticação.

Severidade	Qtd.	Resumo
CRÍTICO	0	Nenhum item crítico identificado.
ALTO	1	[CORRIGIDO] IDOR: instrutor exportava PDF de avaliação de qualquer aluno.
MÉDIO	2	Rate limit em memória (multi-worker); token em localStorage + CSP report-only.
BAIXO	3	Política de senha fraca; bucket público; uso de .single() em caminhos de 'não encontrado'.
INFO	3	CORS em produção; leitura de planos por aluno; dependência externa do Gravatar no cadastro.

Conclusão executiva: o IDOR do PDF de avaliações (único item de severidade Alta) **foi corrigido** nesta revisão, com testes de regressão. A prioridade restante é tratar o rate limit antes de subir em produção com múltiplos workers. O demais são melhorias incrementais de robustez. A base de segurança é confiável.

2. Arquitetura de Autenticação e Autorização

Autenticação: JWT emitido pelo Supabase Auth. O token é validado a cada requisição chamando o Supabase (`auth.get_user(token)`), o que respeita revogação no servidor — e não apenas a verificação local da assinatura. Contas desativadas perdem o acesso tanto no login quanto no `require_auth`, mesmo com token ainda válido.

Autorização por papel: decorators `@require_auth` e `@require_role(...)` aplicam o controle de acesso. Os papéis são *admin*, *receptionista*, *instrutor* e *aluno*.

Ponto-chave (e risco estrutural bem tratado): o backend acessa o banco com a chave `service_role`, que ignora o RLS do PostgreSQL. Ou seja, a autorização real vive na aplicação (decorators) — as policies de RLS são defesa em profundidade. Para as leituras sensíveis por ID, o código abandona a `service_role` e usa `get_user_client(token)`, que reativa o RLS sob a identidade do usuário e fecha BOLA/IDOR (o banco decide quais linhas o usuário vê).

Isolamento de sessão: o login usa um cliente anônimo isolado por requisição, evitando que a sessão de um usuário contamine o cliente global de `service_role` compartilhado entre requisições do Flask — uma armadilha comum e corretamente evitada aqui.

Defesas transversais presentes

- Validação estrita de entrada com Pydantic (`extra="forbid"` → mitiga mass-assignment); faixas numéricas espelham os CHECKs do banco.
- Cabeçalhos de segurança em toda resposta: `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, CSP restritiva na API e HSTS sob HTTPS.
- Respostas de erro 500 não vazam `str(e)` (apenas em modo debug).
- Upload de imagem sempre re-encodado com Pillow (descarta EXIF/polyglots); limite de tamanho em 3 camadas; caminho no Storage isolado por `user_id`.
- Troca de senha revalida a senha atual — token roubado não basta para trocá-la.
- Rate limiting via Flask-Limiter (limite específico e mais rígido no `/login`).
- CORS sem wildcard (origens explícitas); fail-fast se faltarem variáveis de ambiente obrigatórias.
- Sanitização do nome de arquivo no Content-Disposition e escape XML no título do PDF (anti-injeção).

3. Mapa de Endpoints e Controle de Acesso

Inventário das 51 rotas, com método, autenticação exigida e observações de escopo. “RLS” indica que a rota usa o cliente sob a identidade do usuário (proteção contra IDOR).

Método	Rota	Acesso	Observação
auth — /auth			
POST	/login	Público	Rate-limit (10/min); brute-force só limitado por IP
POST	/logout	Autenticado	Revoga a sessão (scope local)
GET/PUT	/me	Autenticado	Próprio perfil
POST	/change-password	Autenticado	Revalida a senha atual
POST/DEL	/me/avatar	Autenticado	Pillow re-encode; pasta = user_id
POST	/me/avatar/gravatar	Autenticado	E-mail vem do backend
alunos — /alunos			
GET/POST		admin, recep.	Listagem paginada; valida entrada
GET	/id	Autenticado	RLS ✓ (fecha BOLA)
PUT	/id	admin, recep.	
PATCH	/id/status	admin, recep.	
GET	/id/planos	Autenticado	RLS ✓
POST	/id/planos	admin, recep.	Bloqueia plano ativo duplicado
GET	/id/planos/v	Autenticado	RLS ✓
PUT/DEL	/id/planos/v	admin, recep.	Exclusão bloqueada se houver paga
instrutores — /instrutores			
GET		admin, recep.	
POST		admin	
GET	/id	Autenticado	RLS ✓ (protege salário)
PUT	/id	admin	
GET	/id/planos	admin, recep.	
POST/DEL	/id/planos[ip]	admin	
planos — /planos			
GET	e /id	Autenticado	Não-admin vê só ativos; dado pouco sensível
POST/PUT/PATCH	/id[/ativo]	admin	
mensalidades — /mensalidades			
GET		admin, recep.	Valida 'mes' antes do banco
GET	/id	Autenticado	RLS ✓
POST	/id/pagar	admin, recep.	Calcula juros; reativa aluno
avaliacoes — /avaliacoes			
GET		admin, instr., recep.	Instrutor: filtrado aos seus alunos
POST		admin, instr.	Ownership do instrutor
GET	/id	admin, instr., recep.	Ownership do instrutor (404)
PUT	/id	admin, instr.	Ownership do instrutor

Método	Rota	Acesso	Observação
DELETE	/ {id}	admin	
GET	/ {id} /pdf	admin, instr., recep.	Ownership do instrutor (404) — corrigido (A-1)
configuracoes — /configuracoes			
GET	/usuarios	admin	Lista profiles + e-mails do Auth
PATCH	/usuarios/{id}/tipo	admin	Não altera o próprio
PATCH	/usuarios/{id}/status	admin	Não desativa a si mesmo
POST/DEL	/usuarios/{id}/avatar	admin, recep.	Foto de terceiros
GET	/academia	Autenticado	Leitura da config
PUT	/academia	admin	
dashboard — /dashboard			
GET	/alunos /financeiro /frequencia	admin, recep.	Agregações
relatorios — /relatorios			
GET	/alunos /financeiro /inadimplencia	admin, recep.	PDF/Excel; valida formato e mês
portal — /portal			
GET	/me /mensalidades /frequencia	Autenticado	Escopo = próprio aluno (profile_id)
GET	/avaliacoes /treino /avisos	Autenticado	Escopo próprio; avisos são broadcast

4. Achados Detalhados

A-1 · IDOR na exportação de PDF de avaliações físicas

CORRIGIDO

ALTO

Local: backend/app/avaliacoes/routes.py — GET /avaliacoes/<id>/pdf (exportar_pdf)

Descrição: O endpoint que devolve a avaliação em JSON (GET /avaliacoes/<id>) verifica o ownership do instrutor — um instrutor só vê avaliações de alunos dos seus planos, recebendo 404 caso contrário. **O endpoint de PDF não faz essa verificação.** Ele usa a service_role (sem RLS) e só exige o papel instrutor. Logo, qualquer instrutor autenticado consegue exportar o PDF de **qualquer** avaliação do sistema, informando apenas o ID.

Impacto: Quebra de isolamento horizontal entre instrutores e exposição de PII e dados de saúde: o PDF contém nome do aluno, **CPF**, peso, percentual de gordura, circunferências e diâmetros corporais de alunos que não pertencem ao instrutor.

Exploração: Trivial — basta estar autenticado como instrutor e iterar/adivinhar IDs (UUID) de avaliações. Não há barreira de RLS porque a rota usa a service_role.

Correção sugerida: Replicar o mesmo guard do endpoint JSON: após carregar a avaliação, se g.user_tipo == 'instrutor' e o aluno_id não estiver em _alunos_do_instrutor(g.user_id), retornar 404.

Status — CORRIGIDO: **Remediado nesta revisão.** O guard de ownership do instrutor foi adicionado ao exportar_pdf (devolve 404 para avaliações fora do escopo do instrutor). Cobertura por 2 testes de regressão em tests/test_avaliacoes.py (instrutor fora do escopo → 404; instrutor do escopo → 200). Suíte completa: 187 testes passando.

M-1 · Rate limit em memória — frágil sob múltiplos workers (produção)

MÉDIO

Local: backend/app/config.py — RATELIMIT_STORAGE_URI = 'memory://'

Descrição: O Flask-Limiter está configurado com armazenamento em memória do processo. O deploy de produção previsto usa gunicorn com Docker — tipicamente vários workers. Com 'memory://', cada worker mantém seu próprio contador, então o limite de 10 logins/min vale por worker: com N workers, o atacante tem efetivamente N×10 tentativas/min. O brute-force protection do /login fica diluído.

Impacto: Enfraquecimento da proteção contra força bruta de credenciais em produção.

Correção sugerida: Definir RATELIMIT_STORAGE_URI=redis://... (storage compartilhado) antes de subir com múltiplos workers, e adicionar o redis ao docker-compose. Já previsto no .env.example; elevar a prioridade dado o deploy iminente.

M-2 · Token JWT em localStorage com CSP ainda em modo report-only

MÉDIO

Local: Frontend (Next.js) — armazenamento de sessão + frontend/proxy.js (CSP)

Descrição: O access token fica no localStorage (acessível por JavaScript). Uma falha de XSS permitiria exfiltrar o token. A mitigação — Content-Security-Policy com nonce — já existe, porém em **modo report-only** (REPORT_ONLY = true), ou seja, ainda não bloqueia, apenas reporta violações.

Impacto: Janela de risco a roubo de sessão caso surja um XSS; a CSP não está efetivamente protegendo enquanto estiver em observação.

Correção sugerida: Validar a CSP tela a tela no navegador e trocar para modo de bloqueio (REPORT_ONLY = false). Avaliar mover o token para cookie httpOnly + SameSite no futuro (exige rever o fluxo de envio do Bearer ao Flask).

B-1 · Política de senha permissiva (mínimo de 6 caracteres)

BAIXO

Local: backend/app/schemas.py — Senha = Field(min_length=6, max_length=128)

Descrição: Senhas de 6 caracteres, sem exigência de complexidade. Aceitável para um MVP, mas baixo para contas administrativas que controlam dados financeiros e PII.

Correção sugerida: Elevar o mínimo para 8+ caracteres e ativar a proteção contra senhas vazadas (leaked password protection) no painel do Supabase Auth.

Local: Supabase Storage — bucket 'avatars' (público) / app/auth/avatar.py

Descrição: As fotos de perfil são servidas por URL pública. O caminho inclui um UUID aleatório (não enumerável), mas qualquer pessoa com a URL acessa a imagem, sem autenticação.

Correção sugerida: Se as fotos forem consideradas sensíveis, migrar para bucket privado com signed URLs de curta duração. Caso contrário, documentar a decisão de mantê-lo público como aceitável.

Local: Vários: avaliaco.es (buscar/atualizar/pdf), mensalidades (pagar), planos (buscar), alunos (vincular_plano), login

Descrição: .single() lança exceção quando não há linha (PGRST116), antes da checagem if not result.data. Em recursos inexistentes, isso resulta em HTTP 500 genérico em vez de 404. Não há vazamento (o handler 500 oculta detalhes), então o impacto é de robustez/consistência, não de exposição.

Correção sugerida: Padronizar para .maybe_single() nesses caminhos de leitura, como já é feito na maior parte do código.

CORS: ALLOWED_ORIGINS tem default localhost. Em produção, definir explicitamente as origens do front; nunca usar wildcard com supports_credentials=True.

Gravatar: O cadastro de aluno e o avatar consultam gravatar.com (GET externo, timeout 5s) — dependência de terceiro no caminho de escrita e leve enumeração de e-mail no Gravatar. Considerar tornar opcional/assíncrono.

Planos: GET /planos/<id> é legível por qualquer autenticado (inclusive aluno) e expõe nomes de instrutores vinculados. Dado pouco sensível — apenas registrar como decisão consciente.

5. Pontos Fortes (o que já está bem feito)

- ✓ **BOLA/IDOR fechado nas leituras por ID sensíveis** (alunos, instrutores, mensalidades) via RLS por identidade — padrão correto e bem aplicado.
- ✓ **Isolamento da service_role**: nunca usada para login; cliente anônimo isolado por requisição evita vazamento de sessão entre requisições.
- ✓ **Validação de entrada estrita** (Pydantic, extra="forbid") em todas as escritas — anti mass-assignment e anti-lixo.
- ✓ **Sanitização de imagem real** com Pillow (re-encode descarta EXIF/polyglots) e limite de tamanho em três camadas.
- ✓ **Troca de senha revalida a senha atual** — token válido roubado não é suficiente.
- ✓ **Autoria de avaliações forçada no servidor** para instrutores (não confia no cliente).
- ✓ **Erros não vazam detalhes internos** (str(e) só em debug); cabeçalhos de segurança padronizados em toda resposta.
- ✓ **Contas desativadas bloqueadas** no login e em cada requisição; fail-fast sem variáveis obrigatórias.
- ✓ **Unicidade garantida no banco** (índices únicos parciais) além da checagem na aplicação — defesa em profundidade contra corrida.
- ✓ **Anti-injeção em downloads**: nome de arquivo saneado no Content-Disposition e escape XML no título do PDF.

6. Plano de Ação Priorizado

Prioridade	Ação	Item
✓ Concluído	Guard de ownership do instrutor adicionado ao GET /avaliacoes/<id>/pdf (com testes).	A-1
1 — Antes do deploy	Configurar Redis como storage do rate limit (compartilhado entre workers).	M-1
2 — Antes do deploy	Definir ALLOWED_ORIGINS de produção; revisar HSTS/HTTPS na borda (Nginx/Cloudflare).	I-1
3 — Curto prazo	Sair do CSP report-only para modo de bloqueio após validação.	M-2
4 — Curto prazo	Elevar mínimo de senha e ativar proteção de senha vazada no Supabase.	B-1
5 — Backlog	Padronizar .maybe_single(); revisar bucket de avatares; Gravatar opcional.	B-2/B-3

Observação metodológica: análise estática (revisão de código-fonte), sem execução de testes de intrusão dinâmicos. Nenhuma alteração foi feita na aplicação. As severidades seguem uma escala qualitativa (Crítico / Alto / Médio / Baixo / Informativo) considerando impacto e facilidade de exploração no contexto do produto.